

Django Module 5 Important

1. Explain the step to deploy a Django project on PythonAnywhere or Heroku, including production settings like DEBUG and ALLOWED_HOSTS.

Deploying a Django Project on PythonAnywhere or Heroku

Deployment is the process of making a Django application available to real users on the internet. Before deployment, Django settings must be changed from development mode to production mode.

Step 1: Configure Production Settings

Disable Debug Mode

In `settings.py`:

```
DEBUG = False
```

- Disables detailed error pages.
- Prevents exposure of source code and sensitive data.
- Displays generic error pages such as 404 and 500.

Configure Allowed Hosts

```
ALLOWED_HOSTS = [  
    'example.com',  
    'www.example.com'  
]
```

- Required when `DEBUG = False`.
- Specifies which domains can access the application.
- Prevents HTTP Host Header attacks.

Step 2: Push Project to Git Repository

```
git init  
git add .  
git commit -m "Initial Commit"
```

Upload the project to GitHub.

Step 3: Deploy on PythonAnywhere

1. Create an account on PythonAnywhere.
 2. Open **Web** tab and create a new web app.
 3. Select **Django** and the required Python version.
 4. Upload or clone the project from GitHub.
 5. Configure the virtual environment.
 6. Set the path to the Django project.
 7. Reload the web application.
-

Step 4: Deploy on Heroku

1. Create a Heroku account.
2. Install Heroku CLI.
3. Login to Heroku:

`heroku login`

4. Create a Heroku application:

`heroku create`

5. Push the project:

`git push heroku main`

6. Run migrations:

`heroku run python manage.py migrate`

Static Files in Production

Static files are not served automatically in production.

Collect static files using:

`python manage.py collectstatic`

Web servers such as **Nginx** or **Apache** serve the collected static files.

Security Settings

Enable production security settings:

```
SECURE_SSL_REDIRECT = True
```

```
CSRF_COOKIE_SECURE = True
```

```
SESSION_COOKIE_SECURE = True
```

These settings improve application security.

Logging and Error Reporting

```
ADMINS = [  
    ('Admin', 'admin@example.com')  
]
```

- Errors can be logged to files or monitoring tools.
- Administrators can receive error notifications.

Production Settings Summary

Setting	Development	Production
DEBUG	True	False
ALLOWED_HOSTS	[]	Domain names
Database	SQLite	PostgreSQL/MySQL
Static Files	Served by Django	Served by Nginx/Apache
Security	Relaxed	Strict

2. Explain XSS, SQL Injection and how Django prevents them.

XSS (Cross-Site Scripting)

XSS is a security attack in which an attacker injects malicious JavaScript code into a web page. When other users open the page, the malicious script executes in their browser.

Example

Suppose a user enters:

```
<script>
alert("Hacked")
</script>
```

If displayed without escaping, the script executes in the browser.

How Django Prevents XSS

Django automatically escapes HTML characters in templates.

```
{{ user_input }}
```

Output:

```
&lt;script&gt;
alert("Hacked")
&lt;/script&gt;
```

The script is displayed as text instead of executing.

SQL Injection

SQL Injection is an attack where malicious SQL code is inserted into user input to manipulate database queries.

Example

Consider the query:

```
SELECT * FROM Student
WHERE name = 'Arun';
```

An attacker enters:

```
' OR '1'='1
```

Resulting query:

```
SELECT * FROM Student
```

```
WHERE name=" OR '1'='1';
```

This may return all records from the table.

How Django Prevents SQL Injection

Django ORM uses parameterized queries automatically.

```
Student.objects.filter(name=user_input)
```

Django safely escapes the input and prevents malicious SQL execution.

Instead of writing raw SQL:

```
Student.objects.raw(
    "SELECT * FROM student WHERE name='%s'" % user_input
)
```

use ORM methods:

```
Student.objects.filter(name=user_input)
```

Security Features in Django

Attack	Description	Django Protection
XSS (Cross-Site Scripting)	Injecting malicious JavaScript into web pages	Automatic HTML escaping in templates
SQL Injection	Injecting malicious SQL commands into queries	Django ORM and parameterized queries

Advantages of Django Security

- Automatic protection against XSS.
- ORM prevents SQL Injection.
- Built-in CSRF protection using `{% csrf_token %}`.
- Secure authentication and session management.

3. Explain how jQuery can be used to make an AJAX GET request in Django.

Using jQuery AJAX GET Request in Django

AJAX (Asynchronous JavaScript and XML) allows data to be sent and received from the server without refreshing the web page. jQuery provides the `$.ajax()` method to perform AJAX requests.

Step 1: Create a Django View

`views.py`

```
from django.http import JsonResponse

def get_data(request):

    data = {
        'message': 'Hello from Django'
    }

    return JsonResponse(data)
```

This view returns data in JSON format.

Step 2: Configure URL

`urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('get-data/', views.get_data),
]
```

Step 3: Create Template

`home.html`

```
<!DOCTYPE html>
<html>
<head>

    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

</head>
<body>
```

```

<button id="btn">
    Get Data
</button>

<h3 id="result"></h3>

<script>

$( '#btn' ).click(function(){

    $.ajax({

        url: '/get-data/',
        type: 'GET',

        success: function(response){

            $( '#result' ).html(
                response.message
            );

        }

    });

});

</script>

</body>
</html>

```

Working

1. User clicks the **Get Data** button.
2. jQuery sends an AJAX GET request to `/get-data/`.
3. Django processes the request in `get_data()`.
4. `JsonResponse` returns data to the browser.
5. jQuery receives the response and displays it without refreshing the page.

AJAX GET Syntax

```

$.ajax({
    url: 'URL',

```

```
type: 'GET',
success: function(response){
    // process response
}
});
```

Advantages

- No page refresh required.
- Faster user experience.
- Data can be loaded dynamically.
- Reduces server load and bandwidth usage.

4. Explain how CORS protection works in AJAX requests in Django.

CORS Protection in AJAX Requests in Django

CORS (Cross-Origin Resource Sharing) is a security mechanism that controls whether a web page from one domain can access resources from another domain.

For example:

Frontend : `http://localhost:3000`

Backend : `http://localhost:8000`

Since the origins are different, the browser blocks AJAX requests unless CORS is enabled.

Why CORS is Needed

Suppose jQuery sends an AJAX request:

```
$.ajax({
    url: 'http://localhost:8000/get-data/',
    type: 'GET'
});
```

The browser checks whether the Django server allows requests from `localhost:3000`.

If not allowed, an error occurs:

Access to XMLHttpRequest blocked by CORS policy

Enabling CORS in Django

Step 1: Install Package

pip install django-cors-headers

Step 2: Add App

settings.py

```
INSTALLED_APPS = [  
    'corsheaders',  
]
```

Step 3: Add Middleware

```
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware',  
    'django.middleware.common.CommonMiddleware',  
]
```

Step 4: Allow Origins

```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:3000",  
]
```

Now requests from `localhost:3000` are allowed.

Example AJAX Request

```
$.ajax({  
    url: 'http://localhost:8000/get-data/',  
    type: 'GET',  
    success: function(response){  
        console.log(response);  
    }  
});
```

Django View

```
from django.http import JsonResponse
```

```
def get_data(request):  
    return JsonResponse({  
        'message': 'Hello Django'  
    })
```

Working of CORS

1. Browser sends AJAX request.
2. Django checks allowed origins.
3. If the origin is listed in `CORS_ALLOWED_ORIGINS`, Django sends:

Access-Control-Allow-Origin:
`http://localhost:3000`

4. Browser allows the response.
5. AJAX receives the data successfully.

Advantages

- Protects against unauthorized cross-origin requests.
- Allows secure communication between frontend and backend.
- Commonly used with React, Angular, Vue, and Django APIs.

5. Explain step-by-step how to integrate a Django REST API with a frontend using Axios, including sending POST requests with JSON data.

Integrating a Django REST API with a Frontend Using Axios

Axios is a JavaScript library used to send HTTP requests to a Django REST API and receive responses.

Step 1: Install Django REST Framework

```
pip install djangorestframework
```

Add it to `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    'rest_framework',  
]
```

Step 2: Create a Model

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
```

Apply migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Step 3: Create Serializer

serializers.py

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = '__all__'
```

Step 4: Create API View

views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Student
from .serializers import StudentSerializer

@api_view(['GET', 'POST'])
def student_api(request):
```

```

if request.method == 'GET':

    students = Student.objects.all()
    serializer = StudentSerializer(
        students,
        many=True
    )

    return Response(serializer.data)

elif request.method == 'POST':

    serializer = StudentSerializer(
        data=request.data
    )

    if serializer.is_valid():
        serializer.save()

    return Response(serializer.data)

```

Step 5: Configure URL

`urls.py`

```

from django.urls import path
from .views import student_api

urlpatterns = [
    path('api/students/', student_api),
]

```

Step 6: Include Axios

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

Step 7: Send GET Request Using Axios

```

<script>

axios.get(
    'http://127.0.0.1:8000/api/students/'

```

```

)
.then(function(response){

    console.log(response.data);

})
.catch(function(error){

    console.log(error);

});

</script>

```

Purpose:

- Retrieves student data from the Django API.
 - Displays the response in the browser.
-

Step 8: Send POST Request with JSON Data

```

<script>

axios.post(
    'http://127.0.0.1:8000/api/students/',
    {
        name: 'Arun',
        email: 'arun@gmail.com'
    }
)
.then(function(response){

    console.log(response.data);

})
.catch(function(error){

    console.log(error);

});

</script>

```

JSON Data Sent

```
{  
  "name": "Arun",  
  "email": "arun@gmail.com"  
}
```

Working

1. User performs an action on the frontend.
2. Axios sends an HTTP request to the Django REST API.
3. Django receives the request.
4. Serializer validates the data.
5. Data is saved to the database.
6. Django returns a JSON response.
7. Axios receives and processes the response.

Advantages of Axios

- Easy HTTP requests.
- Supports GET, POST, PUT, DELETE.
- Automatically handles JSON data.
- Works well with Django REST APIs.
- Supports promises and error handling.

6. Write a Django view that returns a list of students as JSON, and explain how you would fetch it in the frontend using Fetch API.

Django View Returning Students as JSON

`views.py`

```
from django.http import JsonResponse  
from .models import Student
```

```
def student_list(request):
```

```
    students = list(  
        Student.objects.values(  
            'student_id',  
            'name',  
            'email',  
            'department'  
        )  
    )
```

```
return JsonResponse(
    students,
    safe=False
)
```

- `values()` retrieves selected fields.
 - `JsonResponse()` converts Python data into JSON format.
 - `safe=False` allows returning a list of objects.
-

URL Configuration

`urls.py`

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path(
        'students/',
        views.student_list
    ),
]
```

JSON Response

```
[
    {
        "student_id": 1,
        "name": "Arun",
        "email": "arun@gmail.com",
        "department": "MCA"
    },
    {
        "student_id": 2,
        "name": "Rahul",
        "email": "rahul@gmail.com",
        "department": "MCA"
    }
]
```

Fetching Data Using Fetch API

```
<script>

fetch(
  'http://127.0.0.1:8000/students/'
)
.then(response => response.json())
.then(data => {

  console.log(data);

})
.catch(error => {

  console.log(error);

});

</script>
```

Display Data in HTML

```
<ul id="studentList"></ul>

<script>

fetch(
  'http://127.0.0.1:8000/students/'
)
.then(response => response.json())
.then(data => {

  let output = "";

  data.forEach(student => {

    output += `
    <li>
      ${student.name}
      - ${student.department}
    </li>`;
  });

  document.getElementById(
    'studentList'
```

```
    ).innerHTML = output;

});

</script>
```

Working

1. User opens the webpage.
2. `fetch()` sends a GET request to the Django URL.
3. Django view retrieves student records.
4. `JsonResponse()` returns data in JSON format.
5. `response.json()` converts JSON into JavaScript objects.
6. Data is displayed on the webpage without refreshing the page.

Fetch API Syntax

```
fetch('URL')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

Advantages

- Retrieves data asynchronously.
- No page refresh required.
- Easy integration with Django APIs.
- Supports JSON data exchange between frontend and backend.

7. Write a step to generate and download a CSV file in Django?

Steps to Generate and Download a CSV File in Django

CSV (Comma Separated Values) files are commonly used to export data from a Django application.

Step 1: Import Required Modules

`views.py`

```
import csv
from django.http import HttpResponse
from .models import Student
```

Step 2: Create a View to Generate CSV

```
import csv
from django.http import HttpResponse
from .models import Student

def download_csv(request):

    response = HttpResponse(
        content_type='text/csv'
    )

    response[
        'Content-Disposition'
    ] = 'attachment; filename="students.csv"'

    writer = csv.writer(response)

    writer.writerow([
        'Student ID',
        'Name',
        'Email',
        'Department'
    ])

    students = Student.objects.all()

    for student in students:

        writer.writerow([
            student.student_id,
            student.name,
            student.email,
            student.department
        ])

    return response
```

Step 3: Configure URL

urls.py

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path(
        'download-csv/',
        views.download_csv
    ),
]
```

Step 4: Add Download Link in Template

```
<a href="/download-csv/">
    Download CSV
</a>
```

Working

1. User clicks the **Download CSV** link.
2. Django executes the `download_csv()` view.
3. `HttpResponse` is created with CSV content type.
4. `csv.writer()` writes data into the response.
5. Student records are fetched from the database.
6. Data is written row by row into the CSV file.
7. Browser automatically downloads `students.csv`.

Generated CSV File

```
Student ID,Name,Email,Department
1,Arun,arun@gmail.com,MCA
2,Rahul,rahul@gmail.com,MCA
3,Anjali,anjali@gmail.com,BCA
```

Purpose

- Export database records.
- Generate reports.
- Download data in spreadsheet-compatible format.
- Easy data sharing and analysis.

8. Explain how CSRF protection works in AJAX requests in Django.

CSRF Protection in AJAX Requests in Django

CSRF (Cross-Site Request Forgery) is a security mechanism that protects Django applications from unauthorized requests made on behalf of authenticated users.

In normal HTML forms, Django uses:

```
{% csrf_token %}
```

For AJAX POST requests, the CSRF token must be sent manually in the request header.

Step 1: Add CSRF Token in Template

```
{% csrf_token %}
```

Django generates a hidden token:

```
<input type="hidden"
      name="csrfmiddlewaretoken"
      value="abc123xyz">
```

Step 2: Retrieve CSRF Token Using JavaScript

```
function getCookie(name) {
    let cookieValue = null;

    let cookies = document.cookie.split(';');

    for(let cookie of cookies) {
        cookie = cookie.trim();

        if(cookie.startsWith(name + '=')) {
            cookieValue = cookie.substring(
                name.length + 1
            );
        }
    }
}
```

```

        break;
    }
}

return cookieValue;
}

const csrftoken =
    getCookie('csrftoken');
```

Step 3: Send AJAX Request with CSRF Token

Using Fetch API

```

fetch('/add-student/', {

    method: 'POST',

    headers: {
        'Content-Type': 'application/json',
        'X-CSRFToken': csrftoken
    },

    body: JSON.stringify({
        name: 'Arun'
    })

})
.then(response => response.json())
.then(data => console.log(data));
```

Using jQuery AJAX

```

$.ajax({

    url: '/add-student/',

    type: 'POST',

    headers: {
        'X-CSRFToken': csrftoken
    },

    data: {
        name: 'Arun'
    },
```

```
    success: function(response) {  
        console.log(response);  
    }  
});
```

Step 4: Django View

```
from django.http import JsonResponse
```

```
def add_student(request):  
  
    if request.method == 'POST':  
  
        return JsonResponse({  
            'message': 'Student Added'  
        })
```

Working

1. Django generates a CSRF token.
2. Browser stores the token in a cookie.
3. JavaScript reads the token from the cookie.
4. AJAX sends the token in the **X-CSRFToken** header.
5. Django verifies the token.
6. If valid, the request is processed.
7. If invalid or missing, Django returns:

403 Forbidden
CSRF verification failed.

Advantages

- Prevents Cross-Site Request Forgery attacks.
- Ensures requests originate from trusted pages.
- Protects sensitive operations such as form submissions, updates, and deletions.
- Integrated with Django's **CsrfViewMiddleware**.